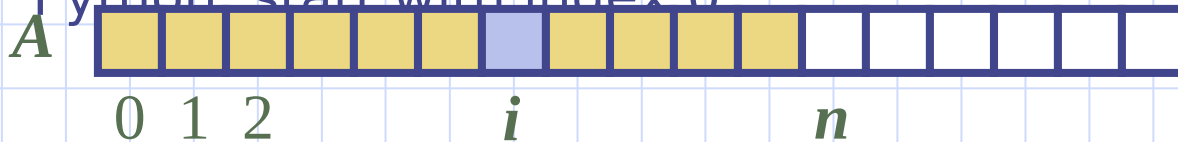


Amortized Analysis



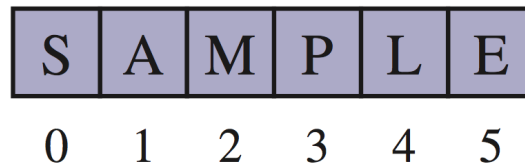
Python Sequence Classes

- ❑ Python has built-in types, **list**, **tuple**, and **str**.
- ❑ Each of these **sequence** types supports indexing to access an individual element of a sequence, using a syntax such as $A[i]$
- ❑ Each of these types uses an **array** to represent the sequence.
 - An array is a set of memory locations that can be addressed using consecutive indices, which, in Python, start with index 0

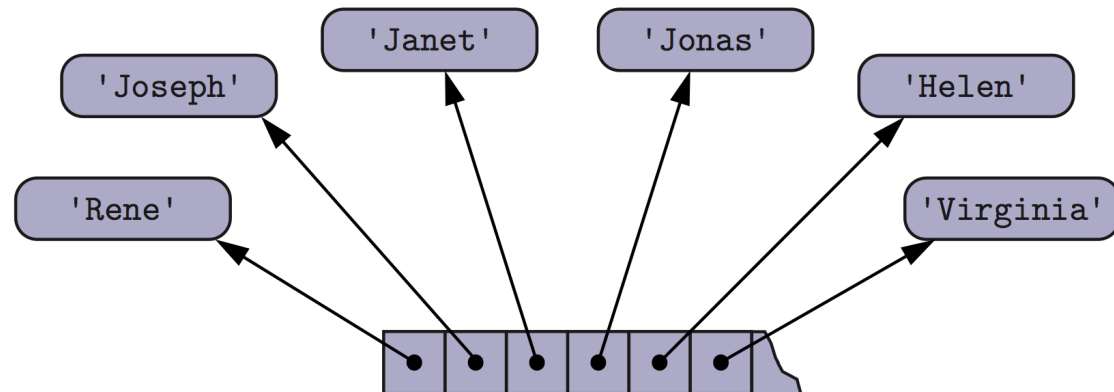


Arrays of Characters or Object References

- An array can store primitive elements, such as characters, giving us a **compact array**.



- An array can also store references to objects.



Amortized Case Analysis

Amortized analysis is a variety of worst case analysis, but rather than looking at the cost of doing the operation once, it examines the cost of repeating the operation in a sequence.

That is, we determine the worst case complexity $T(n)$ of performing a sequence of n operations, and report the amortized complexity as $T(n)/n$.

An alternative view is the accounting method: determine the individual cost of each operation, including both its execution time and its influence on the running time of future operations.

The analogy: imagine that when you perform fast operations you deposit some “time” into a savings account that you can use when you run a slower operation.

Performance

- In an array based implementation of a dynamic list:
 - The space used by the data structure is $O(n)$
 - Indexing the element at i takes $O(1)$ time
 - **add** and **remove** run in $O(n)$ time in worst case
- In an **add** operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one...

Growable Array-based Array List

- In an **add(o)** operation (without an index), we could always add at the end
- When the array is full, we replace the array with a larger one
- How large should the new array be?
 - **Incremental strategy:** increase the size by a constant c

```
Algorithm add(o)  
  if  $t = S.length - 1$  then  
     $A \leftarrow$  new array of size ...  
    for  $i \leftarrow 0$  to  $n-1$  do  
       $A[i] \leftarrow S[i]$   
     $S \leftarrow A$   
   $n \leftarrow n + 1$   
   $S[n-1] \leftarrow o$ 
```

Comparison of the Strategies

- We compare the incremental strategy and the doubling strategy by analyzing the total time $T(n)$ needed to perform a series of n `add(o)` operations
- We assume that we start with an array of size 1
- We call amortized time of an `add` operation the average time taken by an `add` over the series of operations, i.e.,

$$T(n)/n$$

Doubling Strategy

	# of Appends (i)	Cost
	0	0
	1	1
	2	2
	3	3
	4	1
	5	5
	6	1
	7	1

Doubling Strategy

$$\begin{aligned}\sum_{i=1}^n c_i &= \sum_{k=0}^{\log n} 2^k + \sum_{i=1}^n 1 \\ &= 2 \cdot 2^{\log n} - 1 + n \\ &= 3n - 1\end{aligned}$$

This is the cost of n appends.
Hence the amortized cost of each append is
 $3n-1/n = 3$ (constant time)

Amortized Analysis for a Multi-delete Stack

A multi-delete stack is the stack ADT with an additional operation:

1. $mPop(i)$: delete the top i elements from the stack

Assuming a linked representation, the obvious way to execute $mPop(i)$ is to perform pop i times.

If each pop takes b time units, $mPop(i)$ will take approximately ib time units — linear in i !

Worst case is nb time units for stack of size n .

But...

Before you can delete i elements, need to (somewhere along the way...) individually insert i elements, which takes i operations and hence ic time for some constant c .

Total for those $i + 1$ operations is $i(c + b)$. The time for i operations is approximately linear in i . The *average* time for each operation

$$\frac{i}{i + 1}(c + b)$$

is approximately constant — independent of i .

More accurate for larger i , which is also where its more important!

$$\left(\lim_{i \rightarrow \infty} \frac{i}{i + 1}(c + b) = c + b \right)$$

This is called an *amortized analysis*. The cost of an expensive operation is amortized over the cheaper ones which *must* accompany it.

The Accounting Method for the Multi-delete Stack

Every time **push** is called we take a constant time (say a) to perform the operation, but we also put a constant amount of time (say b) in our “time-bank”. When it comes time to perform multi-pop $mPop(i)$, if there are i items to delete, we must have at least ib time units in the bank.

